

API de Gestão Financeira

Arquitetura em camadas, modelo de dados UML e diagramas C4

Resumo do projeto	Descrição
Objetivo	Disponibilizar uma API REST para gerenciar contas, categorias, receitas, despesas e orçamentos pessoais.
Stack	NestJS, TypeScript, TypeORM, PostgreSQL 16, Redis 7 e Docker Compose.
Contrato	OpenAPI/Swagger em /docs; endpoints versionados sob /api/v1.
Arquitetura	Arquitetura em camadas, organizada em módulos por domínio.

Documento de arquitetura e implementação

1. Visão geral

A API de Gestão Financeira oferece operações autenticadas para organizar contas e registrar movimentações financeiras. Cada cliente acessa apenas os próprios dados. Administradores podem consultar a lista de usuários, e o provisionamento administrativo é feito por configuração de ambiente.

Em termos simples

Pense na API como o serviço que fica nos bastidores de um aplicativo financeiro. Ela recebe pedidos como “criar uma conta” ou “registrar uma despesa”, confere quem está fazendo o pedido e guarda as informações nos bancos de dados. Este projeto implementa esse serviço e sua documentação; não inclui uma tela de aplicativo para o cliente final.

Objetivos

- Registrar receitas e despesas vinculadas a contas e categorias.
- Definir limites mensais de orçamento, opcionais por categoria.
- Documentar o contrato HTTP e os modelos usados nas requisições.
- Aplicar autenticação, autorização por papel, idempotência, limitação de requisições, erros padronizados e correlação de logs.

Escopo funcional

Módulo	Responsabilidade
Autenticação	Cadastro de cliente, login JWT e autorização admin/cliente.
Contas	Criar, listar e remover contas financeiras do usuário autenticado.
Categorias	Criar e listar categorias de receita ou despesa.
Transações	Criar, listar com filtros de data e remover movimentações.
Orçamentos	Criar e listar limites mensais gerais ou associados a categorias.
Saúde	Verificar disponibilidade da API e conectividade com PostgreSQL.

Os endpoints de criação cobrem contas, categorias, transações e orçamentos. Não há endpoint de pagamento ou de atualização de conta nesta versão.

2. Arquitetura e tecnologias

A arquitetura escolhida é em camadas. Os módulos são agrupados por domínio para localizar regras e rotas relacionadas, enquanto a separação de apresentação, aplicação e infraestrutura reduz o acoplamento entre transporte HTTP, regras financeiras e persistência.

Camada	Responsabilidades	Implementação
Apresentação	Receber requisições, validar e serializar entradas e saídas.	Controllers, DTOs, class-validator, Swagger
Aplicação	Executar casos de uso e aplicar regras do domínio.	AuthService, AccountsService, TransactionsService e módulos financeiros
Infraestrutura	Persistir dados e manter estado operacional efêmero.	TypeORM/PostgreSQL e ioredis/Redis
Transversal	Aplicar políticas que atravessam os domínios.	JWT/RBAC, filtro RFC 7807, Correlation-ID e rate limiting

Tecnologias

Tecnologia	Uso no projeto
NestJS + TypeScript	Framework HTTP, módulos e injeção de dependências.
TypeORM + PostgreSQL 16	Mapeamento relacional e persistência financeira.
Redis 7 + ioredis	Armazenamento de respostas e chaves de idempotência por 24 horas.
JWT + Passport + bcrypt	Autenticação Bearer, estratégia JWT e hash de senha.
Swagger / OpenAPI	Contrato navegável e execução de chamadas de teste.
Docker Compose	Execução local da API e dos dois serviços de dados.

Organização do código

src/modules contém os módulos de autenticação, contas, categorias, transações, orçamentos e saúde. src/common contém componentes transversais como idempotência, contexto de requisição, autorização por papéis e o filtro global de erros.

3. C4 — Diagrama de Contêineres

O diagrama mostra os elementos executáveis e os limites de responsabilidade: cliente, API NestJS, PostgreSQL e Redis. Swagger UI é servido pela própria API em /docs. Em desenvolvimento local a conexão HTTP pode não usar TLS; em produção, TLS deve ser terminado no ingresso ou balanceador.

Como ler: cada caixa representa um programa ou serviço que participa do sistema; as setas mostram para onde as informações são enviadas. O PostgreSQL é o arquivo organizado dos registros financeiros. O Redis é um apoio rápido para reconhecer pedidos repetidos.

C4 | Diagrama de Contêineres

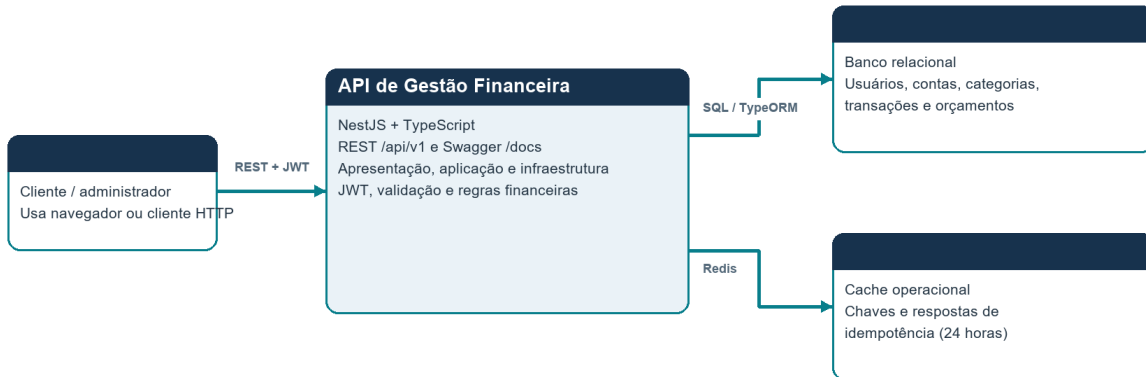


Figura 1. Contêineres e protocolos de comunicação do sistema.

Justificativa dos bancos

- PostgreSQL guarda dados relacionais com integridade referencial: usuários, contas, categorias, transações e orçamentos. Valores monetários usam NUMERIC(14,2).
- Redis mantém respostas idempotentes com expiração e reduz a necessidade de consultar o banco para repetir uma criação já concluída. Não é a fonte dos registros financeiros.
- Docker Compose coordena serviços, health checks e volumes persistentes para desenvolvimento e demonstração local.

4. C4 — Diagrama de Componentes

Dentro do contêiner NestJS, a apresentação direciona chamadas para os módulos de autenticação e financeiro. Componentes transversais aplicam políticas de erro, correlação, idempotência e vazão. Os módulos financeiros usam TypeORM; categorias e orçamentos acessam repositórios diretamente nos controllers nesta implementação.

Em linguagem simples, este desenho abre a caixa da API e mostra suas partes internas. Os componentes da esquerda recebem os pedidos, os do centro aplicam as regras e os da direita conversam com os bancos. A seta dos serviços transversais representa especificamente o uso do Redis pela idempotência; logs são enviados à saída do container e o limitador atual mantém contadores por processo.

C4 | Diagrama de Componentes da API

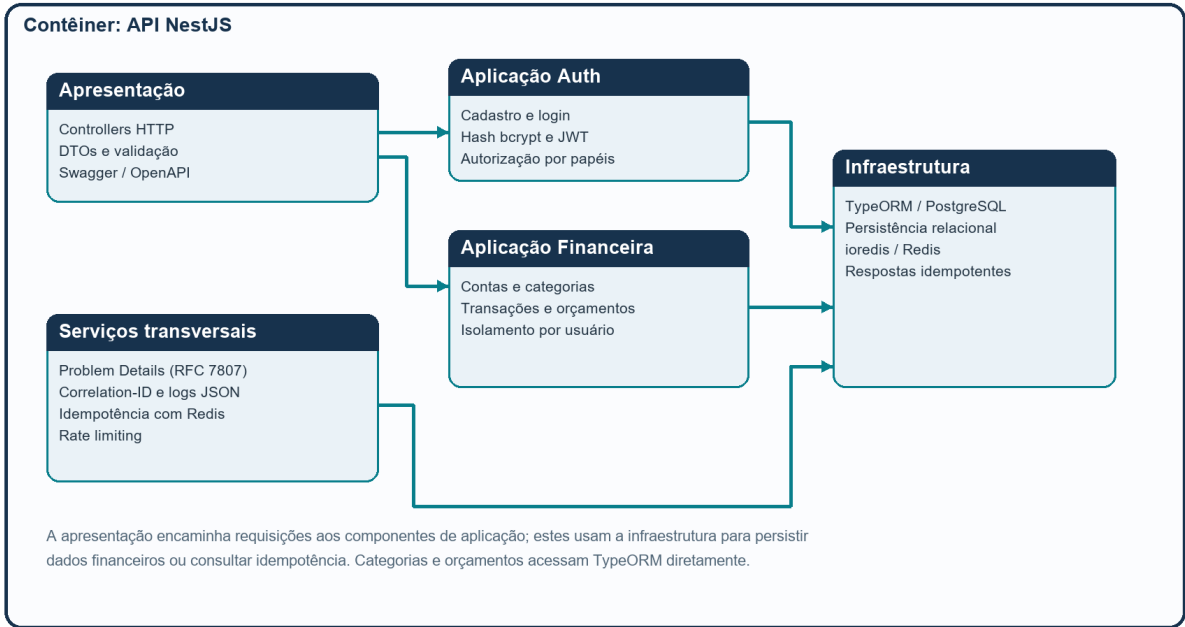


Figura 2. Componentes principais e dependências internas da API.

Fluxo de uma requisição protegida

- O middleware aceita ou gera um Correlation-ID e registra o resultado HTTP como JSON.
- Passport valida o JWT; o papel e o identificador do usuário autenticado ficam disponíveis à rota.
- O controller valida o DTO e encaminha o caso de uso ou consulta ao repositório.
- Criações críticas consultam Redis antes de executar e armazenam a resposta vinculada à chave e ao corpo da requisição.
- O filtro global converte exceções HTTP em application/problem+json.

5. Modelo de dados UML

O modelo descreve as entidades e associações implementadas nos arquivos TypeORM. Um usuário possui seus próprios registros; cada transação pertence a uma conta e pode ter uma categoria; um orçamento pode limitar uma categoria específica ou ser geral.

No UML, cada retângulo representa um tipo de registro e lista algumas informações guardadas. As ligações representam relações: por exemplo, uma conta pode ter várias transações; uma transação pertence a uma conta e pode ter uma categoria. A versão textual abaixo do desenho explica as quantidades permitidas.

UML | Diagrama de Classes do Domínio Financeiro

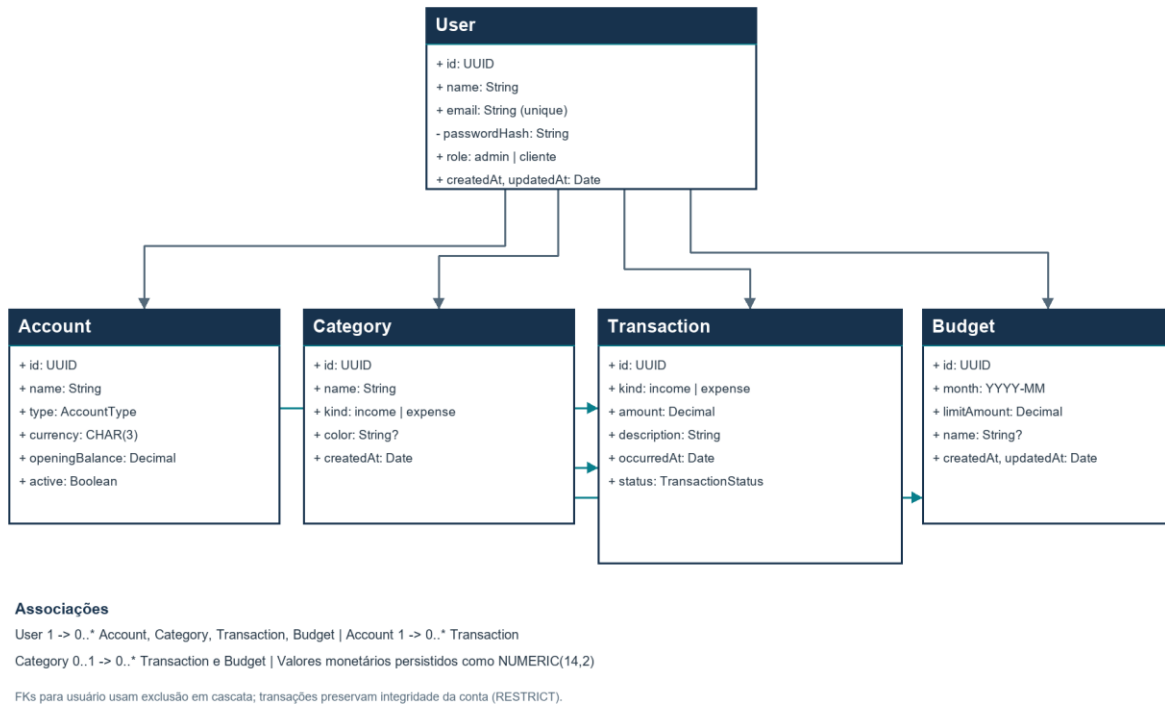


Figura 3. Diagrama UML de classes do domínio financeiro.

Restrições e integridade

- E-mail de usuário é único; conta, categoria e orçamento têm índices compostos para evitar duplicações nos escopos definidos.
- Transação exige uma conta pertencente ao mesmo usuário. Categoria, quando informada, também deve pertencer ao usuário e corresponder ao tipo receita/despesa.
- A conta não pode ser removida enquanto houver transações relacionadas (integridade RESTRICT). A exclusão de usuário remove seus dados dependentes conforme cascatas definidas.
- Valores persistidos como decimal no PostgreSQL são representados como strings pelos campos numéricos do TypeORM; entradas HTTP são validadas com até duas casas decimais.

6. Contrato da API

Base local: <http://localhost:3000/api/v1>. A documentação OpenAPI é publicada em <http://localhost:3000/docs>. O prefixo e as credenciais podem ser configurados por variáveis de ambiente.

Método e rota	Acesso	Descrição
POST /auth/register	Público	Registrar cliente; senha mínima de 8 caracteres.
POST /auth/login	Público	Validar credenciais e devolver token de acesso JWT.
GET /auth/admin/users	Admin	Listar usuários sem retornar hash de senha.
GET /health	Público	Verificar API e PostgreSQL.
GET/POST /accounts	JWT	Listar e criar contas; criação requer Idempotency-Key.
DELETE /accounts/{id}	JWT	Remover conta do usuário autenticado.
GET/POST /categories	JWT	Listar e criar categorias; criação requer Idempotency-Key.
GET/POST /transactions	JWT	Listar com filtros from/to e registrar movimentação idempotente.
DELETE /transactions/{id}	JWT	Remover transação do usuário autenticado.
GET/POST /budgets	JWT	Listar e criar orçamento mensal idempotente.

Exemplo de transação

```
{
  "kind": "expense",
  "amount": 45.90,
  "description": "Almoço",
  "accountId": "UUID da conta",
  "categoryId": "UUID da categoria (opcional)",
  "occurredAt": "2026-09-27",
  "status": "completed"
}
```

As datas de occurredAt usam formato ISO AAAA-MM-DD. Os filtros from e to aceitam o mesmo formato e incluem as datas informadas.

7. Requisitos não funcionais

Requisito	Implementação	Como demonstrar
Autenticação	JWT Bearer; senhas com bcrypt.	Cadastro/login; rota protegida sem token deve responder 401.
Autorização	Papéis admin e cliente; usuários isolados por ID.	Cliente acessa seus dados; cliente em /auth/admin/users recebe 403.
Idempotência	Header Idempotency-Key obrigatório em POST de contas, categorias, transações e orçamentos. Redis guarda a resposta por 24 horas.	Mesma chave/corpo reproduz a resposta; mesma chave/corpo diferente retorna 409.
Erros	Filtro global no formato RFC 7807 application/problem+json.	Enviar DTO inválido e verificar status, title, detail, instance e requestId.
Observabilidade	Correlation-ID/X-Request-ID propagado na resposta e log JSON por requisição.	Enviar Correlation-ID em GET /health e localizar o valor no cabeçalho e logs.
Rate limiting	5 requisições por minuto em login/registro; limite global de 100/minuto.	Exceder o limite produz HTTP 429. O armazenamento atual é por processo.
OpenAPI	Swagger UI em /docs com autenticação Bearer e parâmetros documentados.	Executar operações no navegador Swagger.

Formato de erro

```
{
  "type": "https://httpstatuses.com/400",
  "title": "Bad Request",
  "status": 400,
  "detail": "...",
  "instance": "/api/v1/...",
  "requestId": "..."
}
```

8. Execução e configuração

Inicialização local

- Copiar .env.example para .env e substituir os segredos de exemplo.
- Executar docker compose up --build na raiz do projeto.
- Acompanhar docker compose ps; PostgreSQL e Redis devem ficar healthy e a API deve iniciar após ambos.
- Abrir Swagger em <http://localhost:3000/docs> e health check em <http://localhost:3000/api/v1/health>.

Variáveis e dados

Variável	Finalidade
JWT_SECRET	Assinar tokens de acesso; usar segredo forte fora do ambiente local.
ADMIN_EMAIL / ADMIN_PASSWORD	Provisionar o administrador inicial; senha com pelo menos 12 caracteres.
DB_HOST / DB_DATABASE / DB_USERNAME / DB_PASSWORD	Configurar PostgreSQL. Dentro do Compose, o host do serviço é postgres.
REDIS_HOST / REDIS_PASSWORD	Configurar Redis. Dentro do Compose, o host do serviço é redis.
DB_SYNCHRONIZE	true facilita a demonstração local; produção deve usar false e migrações versionadas.

Limites conhecidos e próximos passos

- Não há fluxo de refresh token implementado nesta versão; um token de acesso expirado requer novo login.
- O rate limiter usa o armazenamento padrão por processo. Para várias réplicas, configurar armazenamento compartilhado Redis ou gateway.
- Antes de produção: criar e executar migrações, configurar backup e restauração, segredos em cofre e TLS no ingresso.
- As rotas oferecem registro de transações financeiras, mas ainda não implementam pagamentos, transferências ou atualização de conta/orçamento.

Glossário

Termo	Explicação sem jargão
API	Serviço que recebe pedidos de um aplicativo e devolve respostas.
Endpoint / rota	Endereço e ação disponíveis, como consultar contas ou registrar uma despesa.
Token JWT	Credencial temporária emitida no login; funciona como um crachá para acessar rotas protegidas.
Idempotency-Key	Identificador de uma tentativa de criação. Repetir a mesma tentativa com os mesmos dados não cria uma cópia.
PostgreSQL	Banco de dados organizado em tabelas, usado como registro permanente dos dados financeiros.
Redis	Banco rápido usado aqui para guardar temporariamente o resultado das tentativas idempotentes.
C4	Modelo de diagramas que apresenta o sistema em níveis: serviços que o compõem e partes internas desses serviços.
UML	Notação visual para mostrar os tipos de dados do projeto e como se relacionam.
DTO	Formato esperado para os campos enviados em uma requisição; a API valida se os dados estão nesse formato.
TypeORM	Ferramenta que traduz entre objetos do código e tabelas do PostgreSQL.